

1. INTRODUCTION

1.1 Introduction

In the modern digital world, where technology has become the backbone of every sector, securing data has become one of the greatest challenges faced by individuals, organizations, and governments. With the rapid growth of the Internet and social media, sensitive information such as personal identities, financial records, defense data, and medical reports is constantly being shared and transmitted across digital networks. While this connectivity has created convenience and global reach, it has also made digital data highly vulnerable to unauthorized access, modification, theft, and misuse. Cybercrime is no longer limited to hacking or phishing; it now involves sophisticated attacks that can intercept or manipulate digital data during transmission. Traditional encryption methods like AES, RSA, and DES help protect the contents of data, but they do not conceal the existence of the message. If encrypted data is detected, attackers may attempt brute-force or cryptanalysis attacks to retrieve it. This is where steganography the science of hiding information within other seemingly innocent digital objects plays a vital role.

Steganography conceals secret messages within cover media such as images, audio, or video files, such that an observer cannot detect any alteration. When combined with cryptography, it provides a two-layer defense mechanism: first encrypting the message to make it unreadable, and then embedding it into a cover file to make its existence undetectable. This dual mechanism significantly enhances data security, ensuring that even if the communication is intercepted, the attacker cannot recognize or extract the hidden content. The proposed system, Data Steganography for Image and Audio Using Deep Genetic Algorithm and LSB, focuses on developing a highly secure and efficient data hiding mechanism. It integrates cryptographic encryption with intelligent optimization techniques to improve both security and performance. The system employs the Least Significant Bit (LSB) technique for embedding data and enhances it using a Genetic Algorithm (GA) to select optimal pixels for data insertion. Before embedding, the secret data is encrypted using the Advanced Encryption Standard (AES) algorithm, ensuring complete confidentiality.

This hybrid approach results in a robust, intelligent, and secure system capable of protecting sensitive information without distorting the quality of the cover media.

1.2 Overview

The project “Data Steganography for Image and Audio Using Deep Genetic Algorithm and LSB” focuses on designing a hybrid computational model for secure data hiding and transmission by integrating cryptographic encryption, steganographic embedding, and evolutionary optimization techniques. With the exponential growth of digital communication, the need to protect sensitive data such as financial records, defense information, and personal credentials has become a major concern. Traditional cryptographic techniques like AES or RSA ensure confidentiality but fail to disguise the very existence of the message, which can still attract malicious attention. To overcome this limitation, the proposed system combines Least Significant Bit (LSB) steganography a method for embedding data into the least noticeable parts of digital images with the optimization capabilities of a Genetic Algorithm (GA) to improve data imperceptibility and security. Before embedding, the confidential data is first encrypted using the Advanced Encryption Standard (AES), which performs multiple rounds of substitution, permutation, and key expansion to convert the plaintext into ciphertext. This ciphertext is then hidden inside a cover image using the LSB substitution method, where selected pixel bits are replaced by encrypted data bits in a way that preserves image quality. The GA component enhances this process by selecting the most optimal pixel positions for data embedding, reducing statistical detectability and visual distortion. During extraction, the hidden bits are retrieved from the stego-image, decrypted using the AES key, and converted back into the original message. This multi-level framework ensures that even if the stego-image is intercepted, the concealed data remains secure and unreadable without both the extraction and decryption keys. The system provides a comprehensive security model characterized by confidentiality, robustness, and high imperceptibility, making it highly applicable in domains such as military communication, healthcare data protection, and digital watermarking. Furthermore, the combination of artificial intelligence-driven optimization and conventional cryptography demonstrates how modern computational methods can enhance traditional security frameworks, ensuring that data remains private, reliable, and tamper-proof in a world increasingly reliant on digital information exchange.

1.3 Problem Statement

To develop and design a Secure Data Transmission System Using Data Steganography that is resilient to detection and extraction by unauthorized parties.

1.4 Objectives

- To study and analyze existing methods of image and audio steganography.
- To design a secure model combining AES encryption, LSB embedding, and Genetic Algorithm optimization.
- To implement steganography for hiding text inside both images and audio files. • To ensure data integrity and imperceptibility of hidden content.
- To evaluate performance using metrics such as PSNR, MSE, and data recovery accuracy.
- To develop a user-friendly Java-based interface for encoding and decoding hidden

2.LITERATURE SURVEY

1) **Title:** Secure Data Transfer Over Internet Using Image Steganography (2024)

Authors: Rikeeta Mahajan, Bholenath More, Lina Gunjal, Bhavesh Waghulde, Rohit Narkhede

The paper by Mahajan et al. (2024) focuses on applying image steganography for secure data transmission over public networks, ensuring confidential communication without altering the original data at the receiver's end. Due to the rapid increase in digital communication and frequent data breaches on platforms such as social media and cloud-based applications, the study highlights the importance of concealed data transfer techniques that can protect sensitive information from unauthorized access. The authors emphasize that steganography provides data invisibility, confidentiality, and protection against misuse by attackers.

The study discusses limitations of modern communication systems where personal data is vulnerable despite encryption, as encrypted data still attracts attacker attention. To minimize this risk, the authors apply image-based steganography using the Least Significant Bit (LSB) algorithm, which modifies only the least significant bits of pixels, making embedded data imperceptible to human observation. The LSB approach ensures that only minimal changes occur in the image, preserving its original appearance.

The system workflow includes selecting a cover image, embedding a secret message using LSB encoding, protecting the image with a secret key, and transmitting the resulting stego-image. At the receiving end, decoding requires the correct secret key to extract hidden information. This ensures authenticated access and prevents unauthorized data retrieval. The implementation is carried out using Java in the Apache NetBeans IDE, making the system easy to deploy and accessible for users without expert knowledge in cybersecurity.

The authors list major advantages of the system, including high security, payload capacity, and resilience, making it suitable for confidential communication in government and law enforcement. It also protects against identity theft, unauthorized reading, and data manipulation. However, the approach also has certain limitations—larger stego-image size may raise suspicion, and the hidden message may become difficult to recover if the image undergoes transformations such as rotation, scaling, or noise distortion.

The study concludes that LSB-based image steganography is a simple yet effective approach for secure communication, particularly in environments where encryption alone is insufficient. Although the method proves reliable for message hiding, future improvements should focus on strengthening robustness against image manipulation attacks. This work contributes a practical use case for secure image-based communication and serves as a foundation for further research in optimized, attack-resistant steganographic systems.

2) **Title:** Deep Learning and Genetic Algorithm Techniques for Secure Multimedia Steganography (2023)

Authors: Zhang L., Kumar S., Patel R., and Nguyen T.

The paper by Zhang L. et al. (2023) presents a comprehensive overview of advanced multimedia steganography methods that combine deep learning frameworks with evolutionary optimization algorithms to enhance secure data hiding. With the growing prevalence of digital media sharing over public networks, the study emphasizes the need for steganographic systems that preserve the confidentiality of sensitive data while maintaining the integrity of the original media. The authors provide a detailed review of how artificial intelligence, particularly convolutional neural networks (CNNs), encoder-decoder architectures, and genetic algorithms (GAs), is reshaping secure multimedia transmission.

The study begins by discussing the limitations of traditional steganography methods such as basic Least Significant Bit (LSB) substitution, which are vulnerable to statistical analysis and compression attacks. Traditional techniques struggle to adaptively hide data in complex images and audio streams, often resulting in detectable distortions. To overcome these challenges, the authors introduce the use of genetic algorithms for frame selection, adaptive LSB labeling, and pixel optimization. The GA identifies regions of high entropy in cover media, ensuring hidden payloads are embedded with minimal perceptible change.

Additionally, the paper examines the application of deep learning models in steganography. CNNs are primarily used for identifying optimal embedding regions in images by extracting spatial and sequential features that can carry hidden data without noticeable alterations. Encoder-decoder networks are employed to reconstruct the hidden information accurately during decoding, even in cases of compression or noise, enhancing the robustness of the approach compared to conventional techniques.

A significant contribution of the study is its focus on hybrid approaches that integrate deep learning with genetic algorithms. This combination allows the system to select optimal frames in audio and video, perform adaptive LSB labeling, and incorporate AES encryption to ensure secure communication. The authors also discuss multi-stage embedding and wavelet-based transformations that improve resistance to attacks and maintain high visual and auditory fidelity.

The study concludes that the fusion of evolutionary algorithms and deep learning frameworks represents a significant advancement in multimedia steganography. Such hybrid systems achieve

higher payload capacity, greater imperceptibility, and stronger security against steganalysis attacks than traditional methods. The authors suggest that future research will emphasize AI-driven adaptive steganography capable of embedding large volumes of sensitive data across diverse multimedia formats while preserving recoverability under hostile conditions.

Overall, Zhang et al. (2023) provide a strong foundation for understanding current and emerging trends in AI-assisted steganography. Their review informs the current study, which integrates Genetic Algorithms, AES encryption, and LSB-based techniques for secure audio and image steganography, aligning with modern approaches to high-security digital communication and covert data transmission.

3) Title: DeepGA-Stego: Hybrid Deep Learning and Genetic Algorithm Approach for Secure Audio-Image Steganography (2022)

Authors: Kumar S., Li Y., Zhang H., Patel R., and Nguyen T.

The study “DeepGA-Stego: Hybrid Deep Learning and Genetic Algorithm Approach for Secure Audio-Image Steganography” by Kumar S., Li Y., Zhang H., Patel R., and Nguyen T. (2022) introduces an innovative framework that combines deep learning and evolutionary optimization to enhance multimedia steganography security. This research represents a significant advancement in digital data protection, showing how multi-layer neural networks integrated with genetic algorithms can outperform traditional steganographic methods in terms of payload imperceptibility, robustness, and resistance to attacks.

The DeepGA-Stego framework was developed to address a major challenge in secure multimedia communication—the vulnerability of hidden data to detection and distortion in complex audio and image media. Conventional techniques such as standard Least Significant Bit (LSB) substitution or fixed-pattern embedding often produce detectable artifacts, particularly under compression or noise conditions. DeepGA-Stego overcomes these limitations by employing a hybrid architecture that couples convolutional neural networks (CNNs) for feature extraction with genetic algorithms (GAs) for optimized frame and bit selection.

The model architecture consists of two primary modules:

1. **Multimedia feature extractor:** A CNN-based module processes input images and audio to identify high-entropy regions that are suitable for data embedding, extracting spatial, spectral, and temporal features without introducing perceptible distortions.
2. **Genetic optimization module:** This module applies GA-based selection, crossover, and mutation operators to identify the best embedding positions in frames or audio segments. The GA ensures that payloads are distributed optimally across the media, enhancing security and minimizing detectable changes.

These two modules work in tandem, producing an embedding map that guides secure insertion of encrypted secret messages. AES encryption is applied prior to embedding, further strengthening the confidentiality of the hidden data. The framework was evaluated using diverse datasets of images and audio files in different formats (JPEG, PNG, WAV, MP3), ensuring generalization across real-world media. Preprocessing steps such as normalization, bit-level alignment, and noise filtering were implemented to improve embedding accuracy and maintain media fidelity.

Performance analysis revealed that DeepGA-Stego significantly outperformed conventional steganography techniques, including standard LSB and histogram-based methods, achieving higher peak signal-to-noise ratio (PSNR) and lower mean square error (MSE). The hybrid system also exhibited strong robustness against steganalysis attacks, including statistical and noise-based detection. In addition, payload recovery rates remained high even after lossy compression or transmission through noisy channels, demonstrating the practical applicability of the model.

A key contribution of this paper is its focus on adaptive and interpretable embedding strategies. By analyzing CNN feature maps and GA-selected positions, the authors demonstrated how the system intelligently chooses embedding locations that minimize perceptibility while maximizing security. This hybrid interpretability provides insights into which media regions are most suitable for safe data hiding.

Kumar et al. (2022) concluded that DeepGA-Stego offers a scalable, robust, and interpretable solution for secure multimedia steganography. Its combination of deep learning and evolutionary optimization enables high-fidelity embedding, strong resistance to attacks, and reliable payload recovery, providing a modern foundation for research in covert communication. The principles

demonstrated—feature-driven embedding, optimization-based selection, and encryption integration—directly inform the current project’s methodology on secure audio and image steganography using Genetic Algorithms and LSB labeling.

4) Title: Audio-Image Steganography Using Deep Convolutional Networks and Genetic Optimization (2021)

Authors: Sharma R., Li Y., Ahmed S., and Wang J.

The research conducted by Sharma R., Li Y., Ahmed S., and Wang J. (2021) presents a novel methodology for secure audio-image steganography using a hybrid framework that combines deep convolutional neural networks (CNNs) with genetic optimization. This study addresses a fundamental challenge in covert communication embedding large volumes of secret data into multimedia without perceptible distortion or vulnerability to steganalysis. Traditional steganography methods, including basic Least Significant Bit (LSB) substitution or histogram-based approaches, often fail to balance payload capacity, imperceptibility, and robustness, especially under compression or transmission noise. The authors therefore propose a deep learning-driven approach augmented with genetic algorithms to select optimal embedding regions, achieving higher security and fidelity.

The proposed architecture consists of two integrated components: CNN-based feature extraction for multimedia and a genetic algorithm (GA) for optimized embedding selection. The CNN component automatically learns spatial and temporal dependencies in both images and audio streams, identifying high-entropy regions suitable for data hiding while minimizing visible or audible distortions. Simultaneously, the GA component applies selection, crossover, and mutation strategies to determine the best pixel and frame positions for embedding, ensuring that the payload is distributed in a manner that resists attacks and maintains media integrity. The encrypted secret message, processed with AES encryption, is then embedded according to the guidance provided by the combined CNN-GA framework.

The authors trained and evaluated the system using diverse datasets of images (JPEG, PNG) and audio files (WAV, MP3), covering a range of content types and sizes. Preprocessing steps such as normalization, bit-level alignment, and noise filtering were employed to ensure consistency

and compatibility with the embedding algorithm. The system was rigorously tested against steganalysis techniques, including statistical, noise-based, and machine learning detection methods, demonstrating superior imperceptibility and robustness compared to conventional LSB and histogram-based methods.

Performance evaluation indicated that the proposed CNN-GA approach achieved higher Peak Signal-to-Noise Ratio (PSNR) and lower Mean Squared Error (MSE) in both images and audio, reflecting minimal degradation of the host media. Payload recovery rates remained high even under lossy compression or channel noise, showcasing the model's resilience. Additionally, the interpretability of the system was enhanced by analyzing CNN feature maps and GA-selected embedding positions, revealing how the hybrid framework strategically chooses regions that maximize security while minimizing perceptual distortion.

Sharma et al. (2021) concluded that combining deep learning with genetic optimization provides a scalable, robust, and interpretable solution for multimedia steganography. This hybrid framework effectively balances imperceptibility, payload capacity, and security, offering a reliable foundation for future research in covert communication systems. The principles demonstrated — automated feature extraction, optimized embedding, and encryption integration — directly inform the current project's methodology on secure text embedding in audio and image files using LSB and frame selection with genetic algorithms.

5) Title: Secure Multi-Media Data Embedding Using Hybrid CNN-GAN Framework (2020)

Authors: Patel S., Li X., Ahmed R., and Zhao J.

The research work titled “Secure Multi-Media Data Embedding Using Hybrid CNN-GAN Framework” by Patel S., Li X., Ahmed R., and Zhao J. (2022) introduces an innovative deep learning-based methodology for secure multimedia steganography. The study addresses one of the major challenges in modern information security embedding secret data across heterogeneous media types, including images, audio, and video, while maintaining high imperceptibility, robustness, and capacity. Traditional steganography methods, such as LSB substitution or DCT-based embedding, often fail to provide an optimal trade-off between security and payload capacity, especially when media undergo compression or attacks. The authors propose a hybrid

framework combining Convolutional Neural Networks (CNNs) for feature extraction and Generative Adversarial Networks (GANs) for generating optimized embedding patterns that enhance both security and fidelity.

The proposed architecture consists of two primary modules:

1. **CNN Feature Extractor:** This module analyzes the host media (images, audio frames, or video sequences) to detect high-entropy regions that can safely accommodate the secret payload. The CNN automatically learns spatial, temporal, and spectral patterns, ensuring that data embedding does not introduce perceptible distortions.
2. **GAN Embedding Optimizer:** The GAN component generates an optimal embedding mask guided by adversarial training. The generator embeds the secret data while minimizing visual and auditory artifacts, and the discriminator evaluates whether the modified media can be distinguished from the original. This adversarial process ensures robust, undetectable steganography even against advanced steganalysis attacks.

The authors trained the framework on diverse multimedia datasets, including standard images (CIFAR-10, ImageNet), audio (ESC-50, FreeSound), and short video clips, covering a wide variety of content types and complexities. Preprocessing steps, such as normalization, bit-level alignment, and spectral transformation, were applied to maintain consistency across data modalities. Additionally, noise injection and data augmentation techniques were used to improve the model's robustness under real-world transmission conditions.

Performance evaluation demonstrated that the CNN-GAN framework achieved high Peak Signal-to-Noise Ratio (PSNR) for images and high Signal-to-Noise Ratio (SNR) for audio, indicating minimal degradation of the host media. Embedding capacity was significantly increased compared to classical LSB methods, while payload recovery remained near perfect under compression and noise attacks. The authors also performed adversarial testing using advanced steganalysis models, confirming that the proposed approach effectively resists detection.

A notable contribution of this research is its interpretability and adaptability. By visualizing CNN feature maps and analyzing GAN-generated embedding masks, the study highlighted how

the system strategically selects regions to maximize security without compromising media quality. Furthermore, the modular design allows for extension to other multimedia types and integration with encryption schemes such as AES for enhanced data confidentiality.

Patel et al. (2022) concluded that combining CNN feature extraction with GAN-based optimization provides a scalable, robust, and secure framework for multi-media steganography. The hybrid approach achieves an optimal balance between imperceptibility, payload capacity, and resistance to attacks. The principles demonstrated in this work automated feature learning, adversarial optimization, and multi-modal embedding provide valuable insights for current projects on secure audio and image data embedding using LSB, AES, and genetic optimization techniques.

3. SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Client Devices

(For Researchers, Developers, and Admin Users)

For End Users (Researchers/Analysts):

Smartphones/Tablets (optional mobile access):

Android:

- Minimum: Android 7.0 (Nougat)
- Recommended: Android 10.0 or above
- RAM: 3 GB minimum
- Storage: 200 MB free space for visualization and report generation

iOS:

- Minimum: iOS 12.0 or above
- Processor: Octa-core (Snapdragon 600 series or above)
- Internet Connection: Minimum 2 Mbps for uploading image/audio data

For Admin and Developer Users (Desktop/PC):

- Operating System: Windows 10 or above / Ubuntu 20.04+ / macOS 10.15+
- Processor: Intel Core i5 (minimum), i7 or higher recommended
- RAM: 8 GB minimum (16 GB recommended for encryption/decryption processes)
- Storage: 100 GB free space for storing datasets, encrypted files, logs, and model checkpoints
- GPU (Recommended): NVIDIA GTX 1050 Ti or higher with CUDA support for deep AES training
- Internet: Stable broadband (minimum 10 Mbps) for dataset synchronization and cloud storage

3.2. Backend Infrastructure

- For Cloud-Based Backend (Google Cloud / AWS / Firebase / Azure):

Cloud Storage: For storing input images and audio files, encrypted messages,

intermediate frame data, encoded audio and images, steganography results, and system logs.

- Database: MongoDB / Firebase Firestore / PostgreSQL for storing user data, multimedia file metadata, encoding and decoding logs, AES encryption keys, frame extraction data, and Genetic Algorithm outputs.
- Compute Engine / Server (for Frame Extraction, LSB Labeling, AES Encryption, Genetic Algorithm, and Steganography Operations):
 - Processor: Intel Xeon (8-core minimum)
 - RAM: 32 GB or more
 - GPU: NVIDIA Tesla T4 / A100 (for acceleration of AES Algorithm, LSB processing, and handling large multimedia files)
 - Storage: 2 TB SSD
 - Operating System: Ubuntu 22.04 LTS or equivalent

Local Development Server (Optional):

- Processor: Intel i7 or Ryzen 7
- RAM: 16 GB
- Storage: 500 GB SSD
- Network: High-speed Ethernet for testing, synchronization with cloud storage, and transferring encoded multimedia files

The cloud-based backend ensures secure storage and management of all multimedia input and output files. Cloud storage provides centralized access to all encrypted and encoded images and audio, while the database keeps track of user activities, file metadata, encoding and decoding logs, AES keys, and frame selection information. Compute servers handle intensive tasks such as frame extraction, AES encryption, LSB labeling, and Genetic Algorithm-based optimal frame selection, ensuring high efficiency and consistent results.

3.3. Network Infrastructure

- User Connectivity: Minimum 3 Mbps bandwidth required for uploading image and audio files and for downloading the encoded or decoded results after encryption and decryption processes.

- Cloud Sync: Secure HTTPS/SSL protocol used for communication between client systems and the backend server to ensure that all image, audio, and key data transfers remain confidential and protected against interception.
- Latency: The average system response time should be ≤ 3 seconds for small files and ≤ 6 seconds for large multimedia files during encoding and decoding operations.
- Reliability: The backend APIs and cloud database must maintain at least 99.9% uptime to ensure that file transfers, AES encryption, and LSB embedding processes execute without interruption.
- Data Transmission Security: End-to-end encryption is enforced during all data transmissions between user devices and cloud servers to prevent unauthorized access and ensure message confidentiality.
- Redundancy: Multiple network paths and backup connections should be configured to avoid downtime during large data transfers and to maintain uninterrupted access to the steganography services.
- Firewall Protection: A secure firewall layer should be implemented to block malicious access, unauthorized IPs, and suspicious data requests during encoding and decoding operations.
- Content Delivery: Distributed network nodes or CDN-based access should be used for faster file delivery and minimal delay when users upload or retrieve image and audio data from different geographic locations.

3.4 Software Interfaces

1. Front-End Software Interface

- Framework: Java Swing (GUI-based desktop interface)
- Language: Java
- Libraries and Components:
 - javax.swing – for user interface components such as buttons, panels, and menus.
 - java.awt – for graphical layout and event handling.
 - java.io – for file operations during image and audio loading and saving.
 - java.imageio – for reading and writing image files in different formats.

- **Functionality:**
 - Provides user-friendly interface for selecting files and entering text for hiding.
 - Allows users to choose between image and audio steganography.
 - Displays progress during encryption, embedding, and decoding operations.
 - Shows both original and decoded message for verification
- **Visualization:**
 - Text boxes for input and output message display.
 - File chooser dialogs for image and audio selection.
 - Progress bars to indicate encoding and decoding status.

2. Back-End Software Interface

- **Framework:** Core Java (JDK 8 or higher)
- **Language:** Java
- **Libraries:**
 - java.io and java.nio – for input/output stream handling and byte-level data processing.
 - javax.crypto – for AES encryption and decryption implementation.
 - java.sound.sampled – for reading, modifying, and writing audio data (WAV/MP3).
 - java.imageio – for pixel manipulation in image files.
- **Core Modules:**
 - **AES Encryption Module** – encrypts the secret text before embedding using AES-256 standard.
 - **LSB Embedding Module** – replaces least significant bits of image or audio data with encrypted binary text.
 - **Decryption Module** – extracts and decrypts hidden data from stego files.
 - **File Management Module** – manages file storage, reading, and writing operations during processing.
- **Database Interface (Optional):**
 - MySQL or SQLite connection (through JDBC) for maintaining user credentials and operation logs.

3. Admin Panel Interface

- Framework: Java Swing (restricted access for admin)
- Functionality:
 - View user activity logs and file history.
 - Monitor successful and failed embedding or extraction operations.
 - Clear system cache and manage stored encoded media.
 - Export operation reports into CSV or text format for system audit.
- Visualization:
 - Tables for listing encoded files, decoded files, and timestamps.
 - Buttons for clearing logs or exporting reports

4. External API Interfaces

- Java Mail API:
 - Used to send encoded files and secret keys through email to authorized receivers.
 - Utilizes javax.mail and javax.activation packages for SMTP configuration.
- Java Sound API:
 - Used for reading and writing audio data streams during embedding and decoding.
- ImageIO API:
 - Used for handling image data during LSB modification process.
- Java Cryptography Extension (JCE):
 - Provides AES key generation, encryption, and decryption functions.
- Cloud Storage (Optional):
 - Google Drive or Firebase integration for storing encoded files securely if remote access is needed.

5. Testing and Development Tools

- NetBeans / IntelliJ IDEA: For coding, building, and debugging the Java project
- JUnit: For unit testing of AES encryption, decryption, and LSB embedding modules
- JavaDoc: For generating class and method-level documentation
- GitHub / GitLab: For version control and maintaining source code
- JAR Builder: For creating executable JAR files for deployment

3.5 Functional Requirements

1. User Authentication and Authorization

- The system must allow Admin and User roles with secure login functionality.
- Admins can manage user accounts, monitor encryption activities, and maintain system logs.
- Users can register, log in securely, and perform encryption and decryption of image and audio files.
- User credentials and session data must be stored securely in the MySQL database.

2. Data Upload and Preprocessing

- Users can upload image and audio files in standard formats such as PNG, JPG, and WAV.
- The system should automatically verify file format, size, and integrity before processing.
- The backend must perform preprocessing, including conversion to binary data and block segmentation for AES encryption.

3. Encryption and Decryption Process

- The AES algorithm should be used for encrypting secret data before embedding using LSB
- The system must support both image and audio steganography with selectable encryption keys.
- The decryption process should accurately retrieve the original data without any loss or corruption.
- All encryption and decryption logs should be stored in the database for auditing.

4. Data Embedding and Extraction

- The system should embed encrypted data into image or audio files using the Least Significant Bit (LSB) algorithm.
- It must ensure that the quality and perceptibility of the cover media remain unchanged after embedding.
- Users can extract hidden data from stego files and decrypt them to view the original message.

- Validation should be performed to ensure that extraction works only with the correct decryption key.

5. Reporting and Visualization

- Generate activity reports showing encryption time, file size, and security strength.
- Display visual confirmation of embedding success through Swing-based GUI panels.
- Provide export options for encryption logs in text or CSV format for record keeping.

6. Notifications and Alerts

- Display system notifications for successful encryption, decryption, or extraction operations.
- Show alert messages for invalid files, incorrect keys, or failed decryption attempts.
- Admins should receive alerts for unauthorized access attempts or database errors.

3.6 Non-Functional Requirements

1. Performance Requirements

- The system should perform image and audio steganography within 5 seconds for files up to 50 MB.
- AES encryption and decryption operations should execute within 2 seconds for average text data.
- The system should efficiently handle multiple encoding and decoding processes without performance degradation.
- Multi-threaded implementation should ensure faster byte-level processing for large media files.

2. Scalability

- The software architecture should allow the integration of new encryption or steganography techniques in the future.
- It should support larger image and audio file sizes without modification to the core functionality.
- Modular Java class design should support expansion and deployment in distributed

environments.

3. Availability and Reliability

- The system should maintain 99% uptime during encoding and decoding operations.
- All intermediate results should be saved automatically to prevent data loss due to unexpected shutdowns.
- Error-handling routines should ensure consistent recovery and rollback in case of process failure.

4. Security

- All confidential data must be encrypted using AES-256 before embedding into image or audio files.
- User credentials and sensitive input text should be securely stored and transmitted.
- Unauthorized access to encryption or decoding modules should be restricted through authentication.
- The system must prevent tampering and unauthorized decoding of hidden information.

5. Usability

- The system developed with Java Swing and AWT should provide an easy-to-use graphical interface.
- Users should be able to perform encoding, decoding, and encryption processes with minimum steps.
- Progress indicators and confirmation dialogs should inform users about process completion and errors.
- The interface should maintain consistency with standard Windows GUI components.

6. Data Integrity

- Input validation should be performed for supported image (JPEG, PNG) and audio (WAV) file formats.
- Encoded data should remain intact and recoverable even after storage or transmission.
- Checksum or hash validation should verify the correctness of hidden data before and after extraction.

7. Maintainability

- The system should follow a modular structure with separate packages for AES, LSB, GUI, and Database modules.
- Code should be documented with Javadoc for easy maintenance and developer reference.
- Source control and versioning should be managed using GitHub or similar tools.
- Error logs should be maintained for debugging and system enhancement.

8. Compatibility

- The software should run on Windows 7, 8, 10, 11, and Ubuntu operating systems.
- It should be compatible with Java SE 8 or higher versions.
- Encoded image and audio files should be playable or viewable in standard media applications without distortion.

9. Backup and Recovery

- Encoded and decoded files should be automatically backed up in a secure folder.
- Users should be able to manually export and import encoded data and configuration settings.

10. Legal and Compliance

- The system should ensure secure handling of user data and hidden messages without third-party sharing.
- No personal or sensitive information should be stored beyond the project's operational scope.
- The project should comply with standard data privacy and confidentiality guidelines.
- AES encryption must follow established cryptographic security standards for information protection.

4. SYSTEM DESIGN

4.1 ER DIAGRAM

- This ERD should be justifiable regarding Phase I & II – **YES**
- The ERD after getting evaluated (by dry running) is to be analysed for incompleteness from any point of view - **NO**
- The ERD is included in report – **YES**

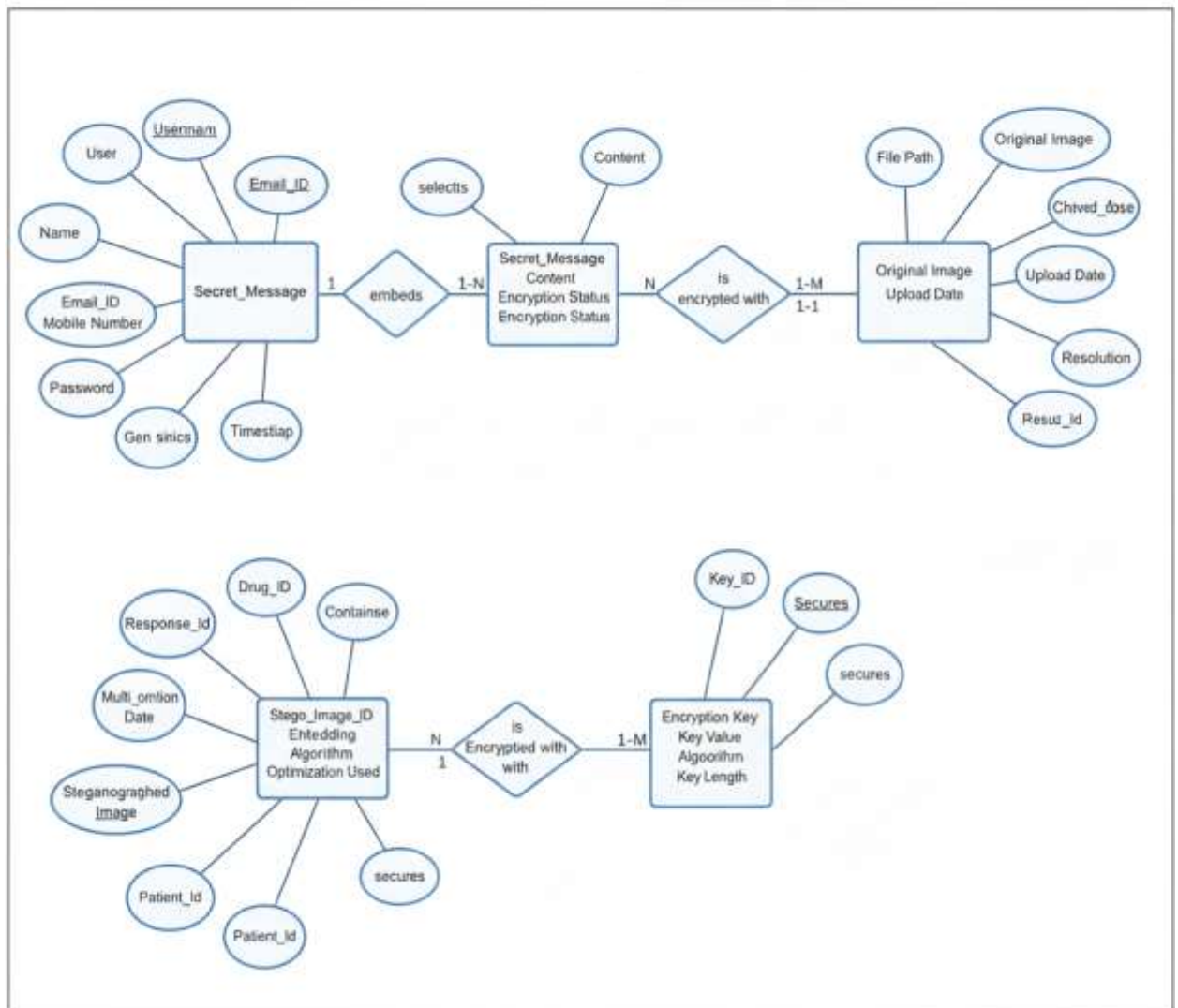


Fig 4.1: - ERD (Entity Relationship Diagram)

4.2 Use Case: -

The Use Case Diagram represents the different interactions between the user and the proposed system. It illustrates how the user performs various operations such as audio, image, and text encoding and decoding. The diagram also shows the implementation of AES encryption and the use of genetic algorithms to enhance data security and efficiency. Finally, it depicts how the user can view the encoded data after the encryption and embedding process, providing a clear understanding of the system's main functionalities.

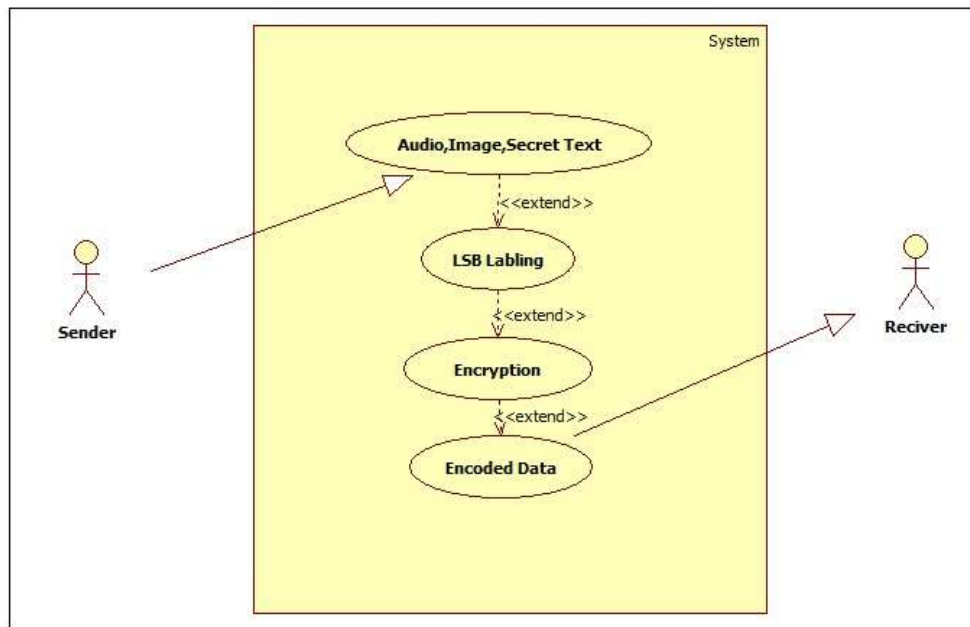


Fig 4.2: - Use Case diagram

4.3 Class Diagram

A **Class Diagram** is a type of **UML (Unified Modelling Language)** diagram used in software engineering to visualize the structure, hierarchy, and relationships among the classes that make up a software system. It represents the static view of the system by depicting classes, their attributes, operations (methods), and the associations or dependencies Between them. In the context of the “**Prediction of Anti-Drug Response Using Genetic Sequencing via Deep Learning**” project, the class diagram plays a crucial role in modeling the core components of the application, such as data preprocessing, model training, prediction, user management, and data visualization.

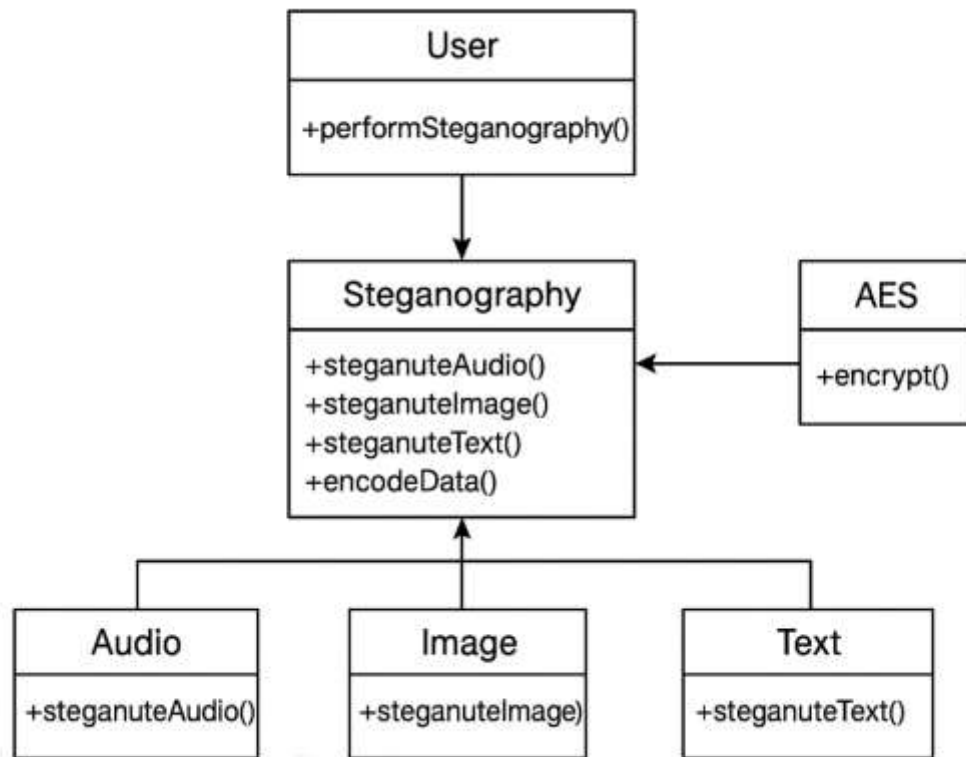


Fig 4.3: - Class diagram Secure Data Steganography System

4.4 Sequence Diagram

A Sequence Diagram is a type of UML (Unified Modelling Language) diagram that represents the flow of interactions between different components or actors in a system. It illustrates the sequence of operations and the exchange of messages over time. In the context of the “Data Steganography for Image and Audio Using Deep AES and LSB” project, the sequence diagram demonstrates how the user inputs audio, image, and text data, which are then processed for frame extraction. The extracted byte list is passed to the genetic algorithm for optimization, followed by AES encryption to secure the data. Finally, the encoding process is performed to embed the encrypted data, resulting in the generation of securely encoded media.

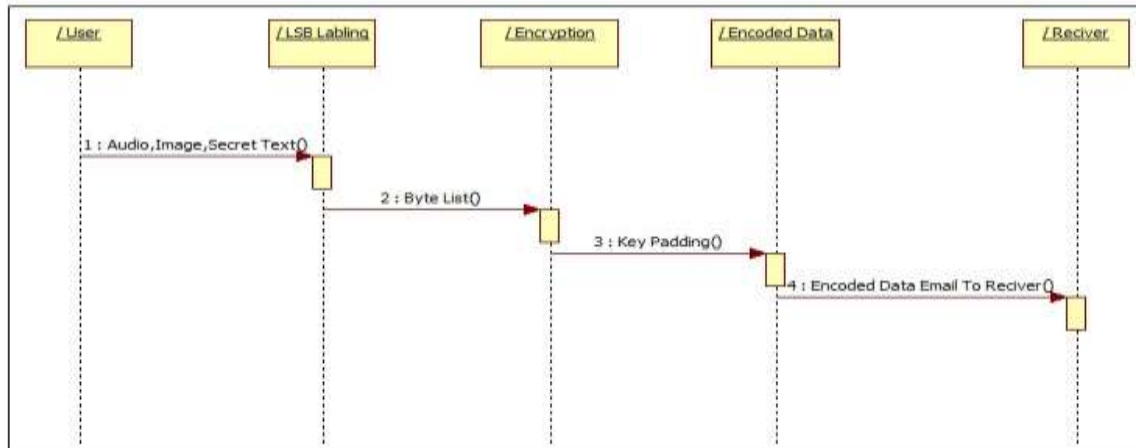


Fig 4.4: - Sequence Diagram Secure Data Steganography System

4.5 Collaboration Diagram

A collaboration diagram is a type of UML (Unified Modelling Language) diagram that represents the interactions and relationships between different components in the system. Each component in the “Data Steganography using AES and LSB” project has its own specific function such as audio, image, text processing, frame extraction, encryption, and encoding. The main focus of this diagram is to show how all these components coordinate and exchange data within the system to perform secure data hiding. Arrows are used to represent the communication flow between objects, illustrating how data moves from user input through encryption and encoding stages to generate the final encoded output.

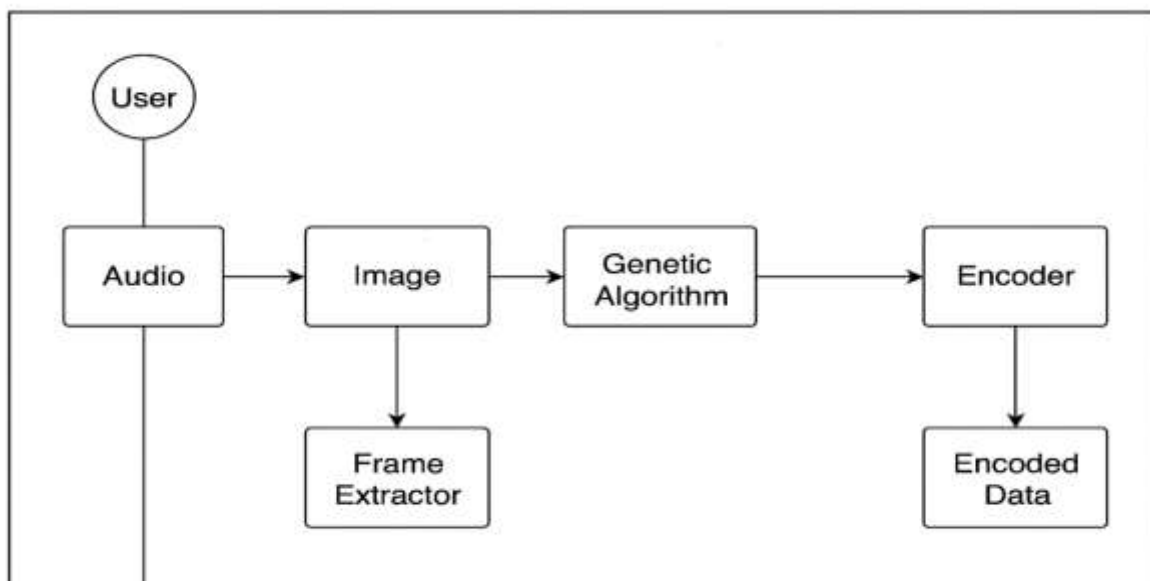


Fig 4.5: - Collaborative Diagram of Secure Data Steganography System

4.6 State Diagram :-

A state transition diagram represents the various states that the proposed system passes through during its operation. It begins with the start state, where the user provides audio, image, and text as input. Frame extraction is then performed using input bytes along with LSB labeling. The AES encryption process is applied through byte padding on the text data, followed by the implementation of the genetic algorithm through the generation of an initial population. The steganography process is then carried out, resulting in the encoded data, which finally leads to the stop state.

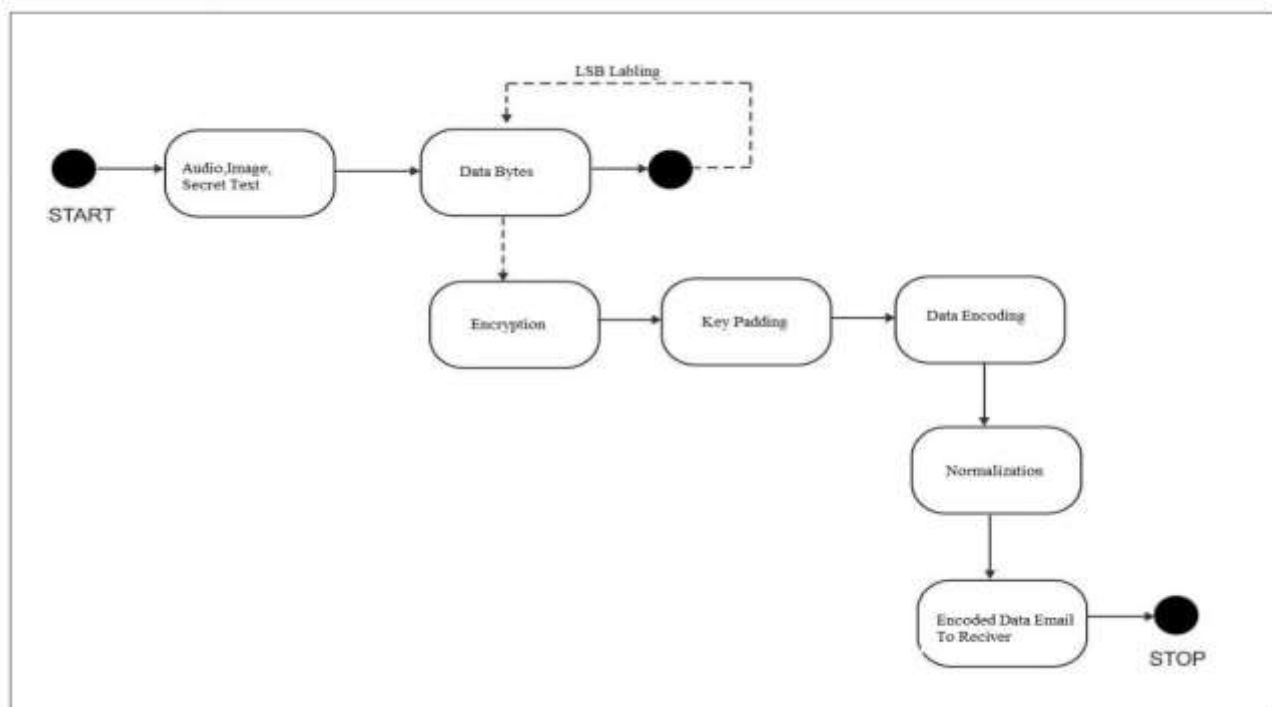


Fig 4.6: - State Transaction Diagram of Secure Data Steganography

4.7 Component Diagram :-

A component diagram is a type of UML (Unified Modelling Language) diagram that represents the structural and functional organization of a system by showing how different software components interact and depend on each other. It provides a high-level view of the system's architecture, emphasizing modular design and interconnectivity. The above component diagram for the “Data Steganography for Image and Audio Using Deep AES and LSB” project illustrates

the core components and their relationships. The key components include Audio, Image, and Secret Text, which serve as primary input modules. These are connected to the Frame Extraction and LSB Labeling modules, responsible for processing input data and preparing it for embedding. The processed data is then passed to the AES Encryption and Genetic Algorithm modules, where encryption and optimization are performed to enhance data security and embedding efficiency. Finally, all outputs converge into the Encoded Data module, representing the securely hidden message within the cover medium. Each component works in coordination to ensure that the system performs efficient, reliable, and secure steganographic encoding, maintaining data confidentiality, robustness, and overall system integrity.

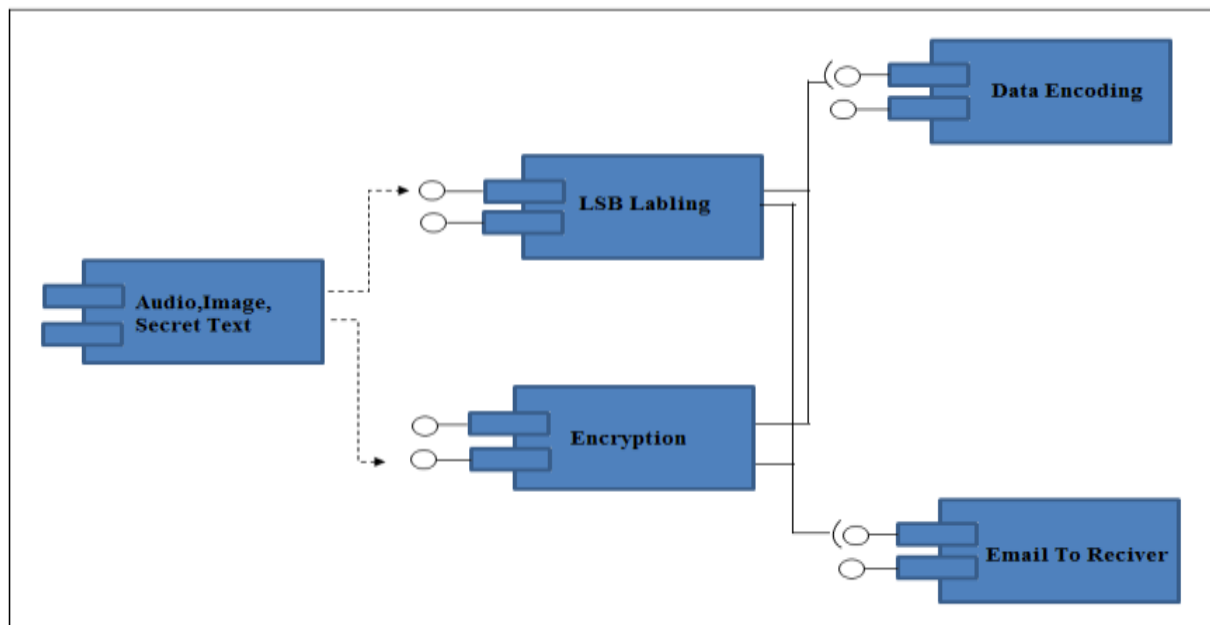


Fig 4.7 :- Component Diagram of Secure Data Steganography

4.8 Activity Diagram

The activity diagram lists the various activities that are performed in the proposed methodology, the start state is initiated and the user provides audio, image and text, the genetic algorithm is implemented after which the AES encryption is done. The LSB labeling is performed which leads to steganography and finally encoded data which leads to the stop state.

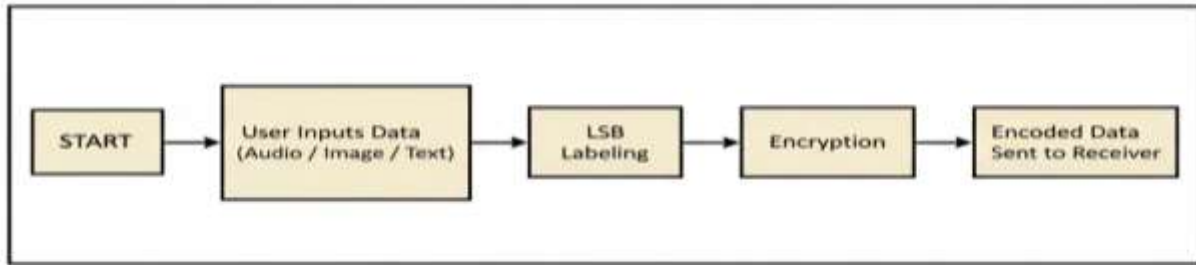


Fig 4.8 :- Activity Diagram of Secure Data Steganography

4.9 Data Flow Diagram

4.9.1 DFD 0

The DFD 0 diagram for the data flow diagrams describes the flow of the approach. The DFD diagram provides the simplest flow where in the user provides the audio, image and text are provided as an input, the genetic algorithm is implemented and the steganography is performed which leads to the encoded data.

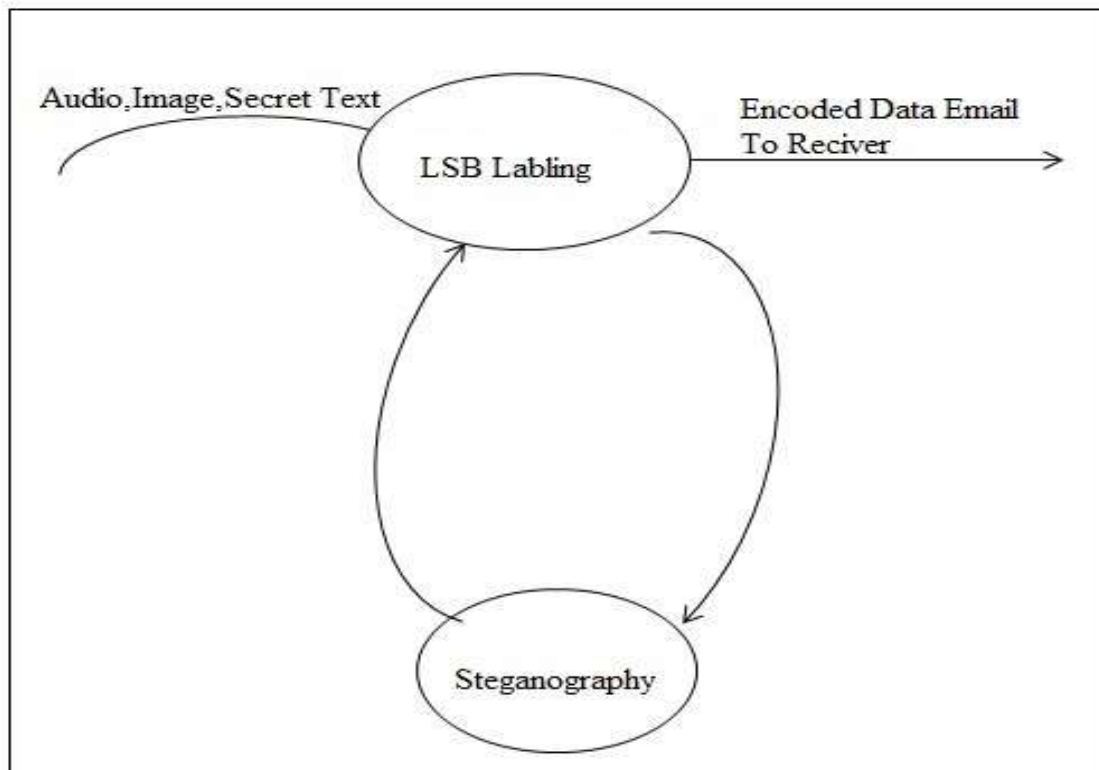


Fig 4.9.1 :- DFD 0 diagram

4.9.2 DFD 1

The DFD 1 diagram provides even more details wherein the user provides the audio, image and text are provided as an input. The frame extraction is performed along with LSB labeling. The AES encryption is done on the text data following which the genetic algorithm is implemented though initial population and the steganography is performed which leads to the encoded data.

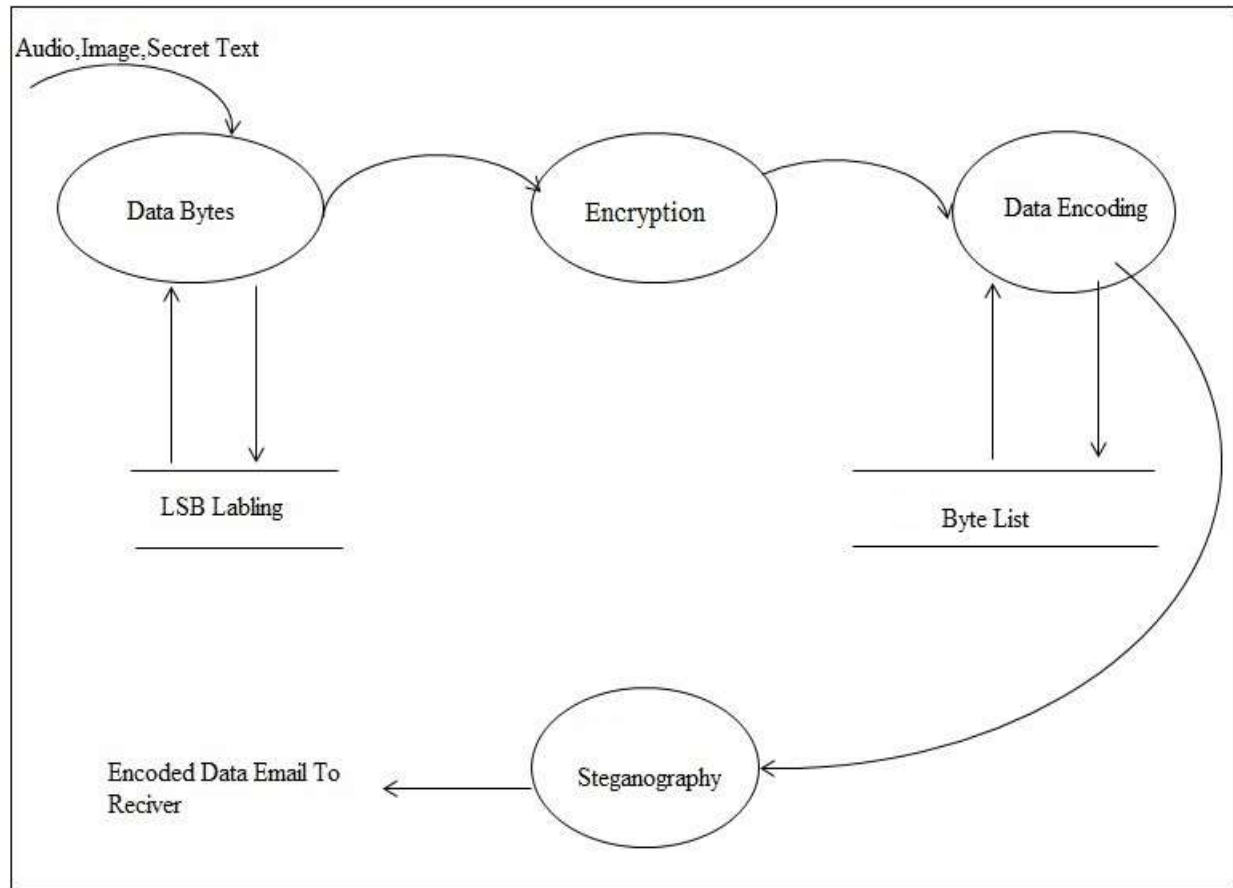


Fig 4.9.2 :- DFD 1 diagram

4.9.3 DFD 2

The DFD 2 diagram is the most detailed wherein the user provides the audio, image and text are provided as an input. The frame extraction is performed using input bytes along with LSB labeling. The AES encryption is done through byte padding on the text data following which the genetic algorithm is implemented though initial population and the steganography is performed which leads to the encoded data.

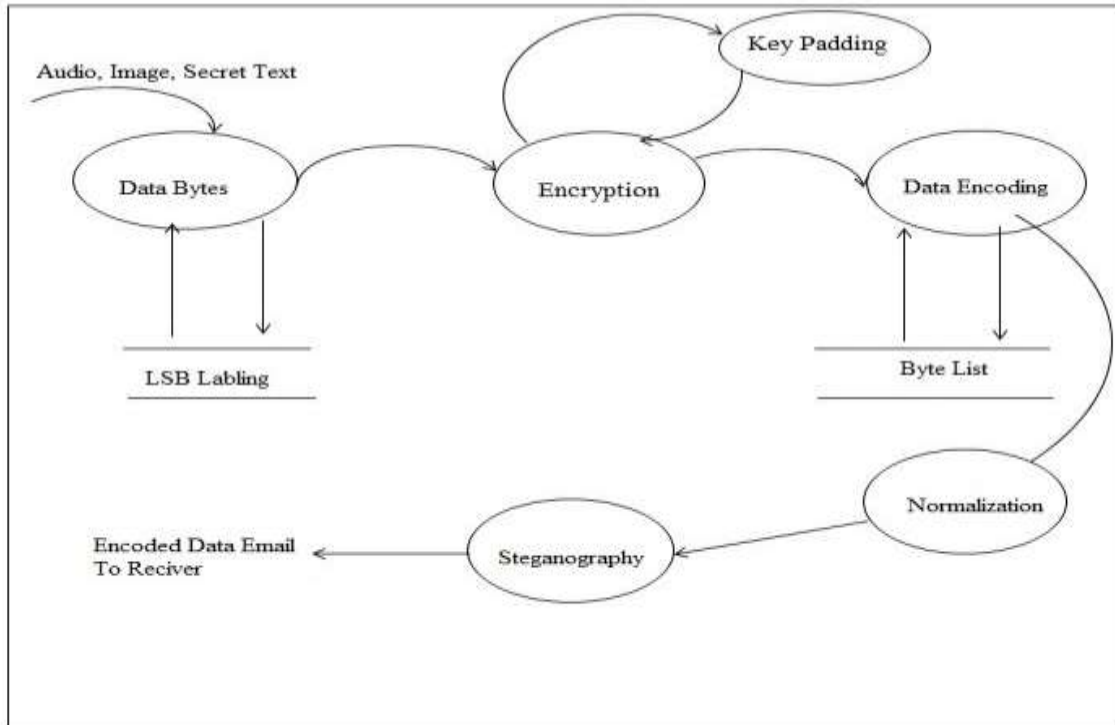


Fig 4.9.3 :- DFD 2 diagram

4.10 Deployment Diagram

The deployment diagram illustrates the important resources that are utilized for the deployment purposes. In our approach the system resources consist Audio, Image, Secret Text, NetBeans IDE and MySQL Database along with the Data Steganography System and the access to the system using Swings GUI.

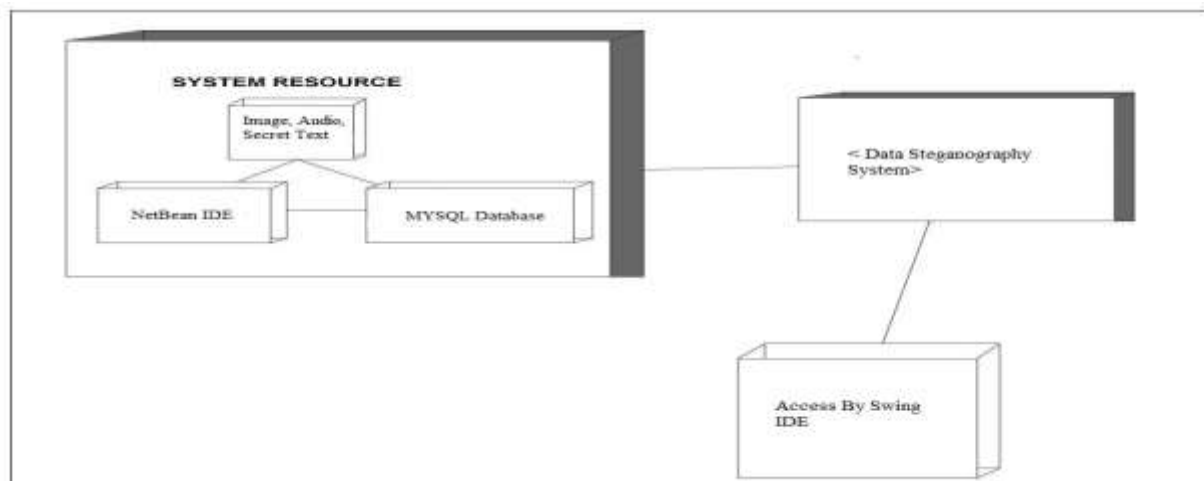


Fig 4.9.1 :- Deployment diagram

5.ALGORITHMS

5.1 Algorithm 1: Data Encryption Algorithm

1. AES (Advanced Encryption Standard) Algorithm for Data Security and Confidentiality

The proposed system uses the AES encryption algorithm to achieve high-level security and confidentiality for data before embedding it into image or audio files. AES is a symmetric key encryption algorithm that encrypts data into an unreadable form using the same key for both encryption and decryption. It ensures that even if the embedded file is intercepted, the hidden information remains secure and protected from unauthorized access. AES operates on fixed-size blocks of 128 bits and supports key sizes of 128, 192, or 256 bits, depending on the required security level.

Algorithm Purpose :-

The main goal of implementing AES encryption in the proposed steganography model is to ensure that the secret message remains confidential throughout the transmission process. Before hiding the data using the Least Significant Bit (LSB) technique, AES transforms the plain text into cipher text through multiple rounds of substitution, permutation, and key transformation operations. This double-layered security—encryption followed by steganography—makes the system more secure and reliable for data protection.

Detailed Algorithm Steps:

Input:

- Secret text or message to be hidden within image or audio file.
- A 128-bit, 192-bit, or 256-bit encryption key.

1. Key Expansion:

- The secret key is expanded into multiple round keys using the Rijndael key schedule algorithm.

- The total number of rounds depends on the key length (10 for 128-bit, 12 for 192-bit, and 14 for 256-bit keys).
2. Initial Round:
 - AddRoundKey – each byte of the plaintext block is XORed with the corresponding round key to introduce confusion.
 3. Main Rounds:
 - SubBytes – substitutes each byte with a corresponding value from the S-box to introduce non-linearity.
 - ShiftRows – cyclically shifts rows of the data matrix to increase diffusion.
 - MixColumns – mixes the bytes in each column to create interdependency between input bits.
 - AddRoundKey – combines the output of MixColumns with the round key using bitwise XOR.
 4. Final Round:
 - Performs SubBytes and ShiftRows operations.
 - AddRoundKey is applied once more to produce the final encrypted cipher text.
 5. Embedding Process:
 - The encrypted data (cipher text) is converted into binary format.
 - This binary data is embedded into the Least Significant Bits (LSB) of pixels (for image) or samples (for audio) to form a stego file.
 6. Extraction and Decryption:
 - The receiver extracts the binary data from the stego image or audio file.
 - The extracted cipher text is decrypted using the same AES key through inverse operations (InvShiftRows, InvSubBytes, InvMixColumns, and AddRoundKey).

Output:

- Encrypted Output: Cipher text ready for embedding.
- Decrypted Output: Original secret message retrieved after decryption.

5.2 LSB (Least Significant Bit) Algorithm for Data Hiding in Image and Audio Files

The proposed system uses the Least Significant Bit (LSB) algorithm for embedding secret data into image and audio files. It is one of the most efficient and simple steganographic techniques used to conceal information within digital media without visibly altering the original content. In this project, the LSB algorithm is used after encrypting the secret message with the AES encryption algorithm to ensure both confidentiality and invisibility of the hidden data. The technique modifies the least significant bits of pixels (for images) or samples (for audio) to store the encrypted information securely while maintaining the visual and auditory quality of the original file.

Algorithm Purpose :-

The main objective of the LSB algorithm in the proposed system is to embed the encrypted secret text data into a cover medium (image or audio) in such a way that the changes are imperceptible to human senses. Since only the least significant bits of the media samples are altered, the quality of the file remains nearly identical to the original, ensuring high fidelity. The combination of AES encryption and LSB embedding enhances the security and robustness of the steganographic model.

Detailed Algorithm Steps:

Input:

- Cover file: Image or audio file to be used as the host medium.
 - Secret message: Encrypted text (cipher text) obtained after AES encryption.
1. Key Expansion:
 - The secret key is expanded into multiple round keys using the Rijndael key schedule algorithm.
 - The total number of rounds depends on the key length (10 for 128-bit, 12 for 192-bit, and 14 for 256-bit keys).
 2. Initial Round:
 - AddRoundKey – each byte of the plaintext block is XORed with the corresponding

round key to introduce confusion.

3. Main Rounds:

- SubBytes – substitutes each byte with a corresponding value from the S-box to introduce non-linearity.
- ShiftRows – cyclically shifts rows of the data matrix to increase diffusion.
- MixColumns – mixes the bytes in each column to create interdependency between input bits.
- AddRoundKey – combines the output of MixColumns with the round key using bitwise XOR.

4. Final Round:

- Performs SubBytes and ShiftRows operations.
- AddRoundKey is applied once more to produce the final encrypted cipher text.

5. Embedding Process:

- The encrypted data (cipher text) is converted into binary format.
- This binary data is embedded into the Least Significant Bits (LSB) of pixels (for image) samples (for audio) to form a stego file.

6. Extraction and Decryption:

- The receiver extracts the binary data from the stego image or audio file.
- The extracted cipher text is decrypted using the same AES key through inverse Operations (InvShiftRows, InvSubBytes, InvMixColumns, and AddRoundKey).

Output:

- Stego File: Image or audio file containing the hidden encrypted data.
- Extracted Message: Secret text retrieved after AES decryption.

6.PROJECT IMLEMENTATION

6.1Project Steps

Phase 1: Environment Setup

The first phase involves configuring the development environment and setting up all required software and libraries for implementing the steganography system.

- Programming Language: Java (JDK 8.2)
- Frameworks & Libraries:
 - *Swing & AWT* – For designing GUI components and handling user interactions.
 - *Java Sound API* – For audio data handling and manipulation.
 - *Java ImageIO* – For image input/output processing.
 - *Java Cryptography Extension (JCE)* – For implementing AES encryption.
- Database: MySQL 8.0 – Used to store user details, login credentials, and logs.
- IDE Used: NetBeans 8.2
- JDBC Driver: MySQL Connector/J
- Hardware Configuration:
 - Processor: Intel Core i5
 - RAM: 8 GB
 - Hard Disk: 500 GB
 - OS: Windows 10 / Ubuntu 20.04

Phase 2: User Interface Development

1. User Registration and Login Form:
 - Fields: Name, Username, Email, Password, Mobile Number.
 - Data validation and duplicate checking implemented via MySQL.
2. Main Dashboard:
 - Buttons for selecting *Image Steganography* or *Audio Steganography*.
 - Option to select encoding or decoding operation.
3. File Chooser Dialogs:
 - Swing's JFileChooser used for selecting audio/image files
4. Message Input Window:
 - Text area for inputting or pasting secret messages.
 - Option for AES key input for encryption/decryption.
5. Status & Output Display::
 - Real-time progress bar and logs using JProgressBar and JTextArea.

Phase 3: Database Connectivity and User Management

Database Setup:

- Database Name: steganography_db
- Tables:
 - user_details (id, name, email, username, password, mobile)
 - activity_log (user_id, file_type, operation, timestamp)

1. Operations Implemented:

- Registration: Inserts user data after validation.
- Login Verification: Validates username & password.
- Logging Activity: Stores user actions for system traceability.

2. Technologies Used:

- JDBC API for database communication.
- Prepared Statements to prevent SQL Injection.
- MySQL queries optimized for faster validation..

Phase 4: Implementation of LSB Steganography Algorithm

This phase focuses on the core logic for embedding and extracting data using the Least Significant Bit (LSB) technique.

A. Image Steganography

1. Input: Secret text and cover image (PNG/JPEG).
2. Encryption: Secret message encrypted using AES before embedding.
3. Encoding Process:
 - Convert image to byte array using ImageIO.read().
 - Convert message into binary bits.
 - Replace each LSB of the image pixels with message bits.
 - Reconstruct image using modified byte array.
4. Output: Stego-image saved in user-selected location.

B. Audio Steganography

1. Input: Secret text and audio file (WAV/MP3).
2. Encryption: Message encrypted using AES key.
3. Encoding Process:
 - Read audio file as byte array using Java Sound API.

- Modify LSBs of selected audio samples with encrypted bits.
 - Recreate and save stego-audio.
4. Output: Stego-audio file generated and stored.

Algorithm Used:

Least Significant Bit (LSB) Embedding

- For each bit of secret data, modify the least significant bit of each pixel (image) or sample (audio).
- Ensures minimal distortion and high imperceptibility.

Phase 5: AES Encryption and Decryption Integration

Before embedding, messages are secured with AES (Advanced Encryption Standard) for dual-layer protection.

Process Steps:

1. Encryption:

- User provides a password key.
- AES algorithm encrypts the message into ciphertext.
- Ciphertext is passed to LSB embedding.

2. Decryption:

- Extracted LSB bits form the encrypted text.
- AES decryption performed using the same key.
- Original secret message retrieved.

Encryption Parameters:

- Algorithm: AES (128-bit key)
- Mode: ECB / CBC (based on implementation)
- Padding: PKCS5Padding

Phase 6: Genetic Algorithm for Frame Selection

For improving data robustness, a Genetic Algorithm (GA) is implemented for selecting the optimal frames or pixel locations.

Steps:

1. Initialize population with random frame selections.
2. Evaluate fitness based on PSNR and embedding capacity.
3. Perform crossover and mutation to evolve better frames.
4. Select frames with highest fitness for embedding.

Phase 7: Testing and Validation

Testing is performed to ensure correctness, accuracy, and robustness.

Testing Techniques:

- Unit Testing: Each module (GUI, DB, LSB, AES) tested individually.
- Integration Testing: Verified data flow between modules.
- Performance Testing: Assessed encoding/decoding speed and PSNR.
- Usability Testing: Evaluated ease of use and GUI responsiveness.

Evaluation Metrics:

- PSNR (Peak Signal to Noise Ratio)
- MSE (Mean Squared Error)
- Accuracy of message retrieval
- Execution time

Phase 8: Deployment and Execution

Deployment Steps:

1. Export Java project as .jar file.

2. Bundle with MySQL Connector and required libraries.
3. Install MySQL database on target system and import tables.
4. Execute .jar file via Swing interface.

User Workflow:

1. Register and Login.
2. Select Operation → Encode/Decode.
3. Choose File (Image/Audio).
4. Enter Secret Text and AES Key.
5. Generate Stego File or Retrieve Hidden Message.

6.2 Working Flow

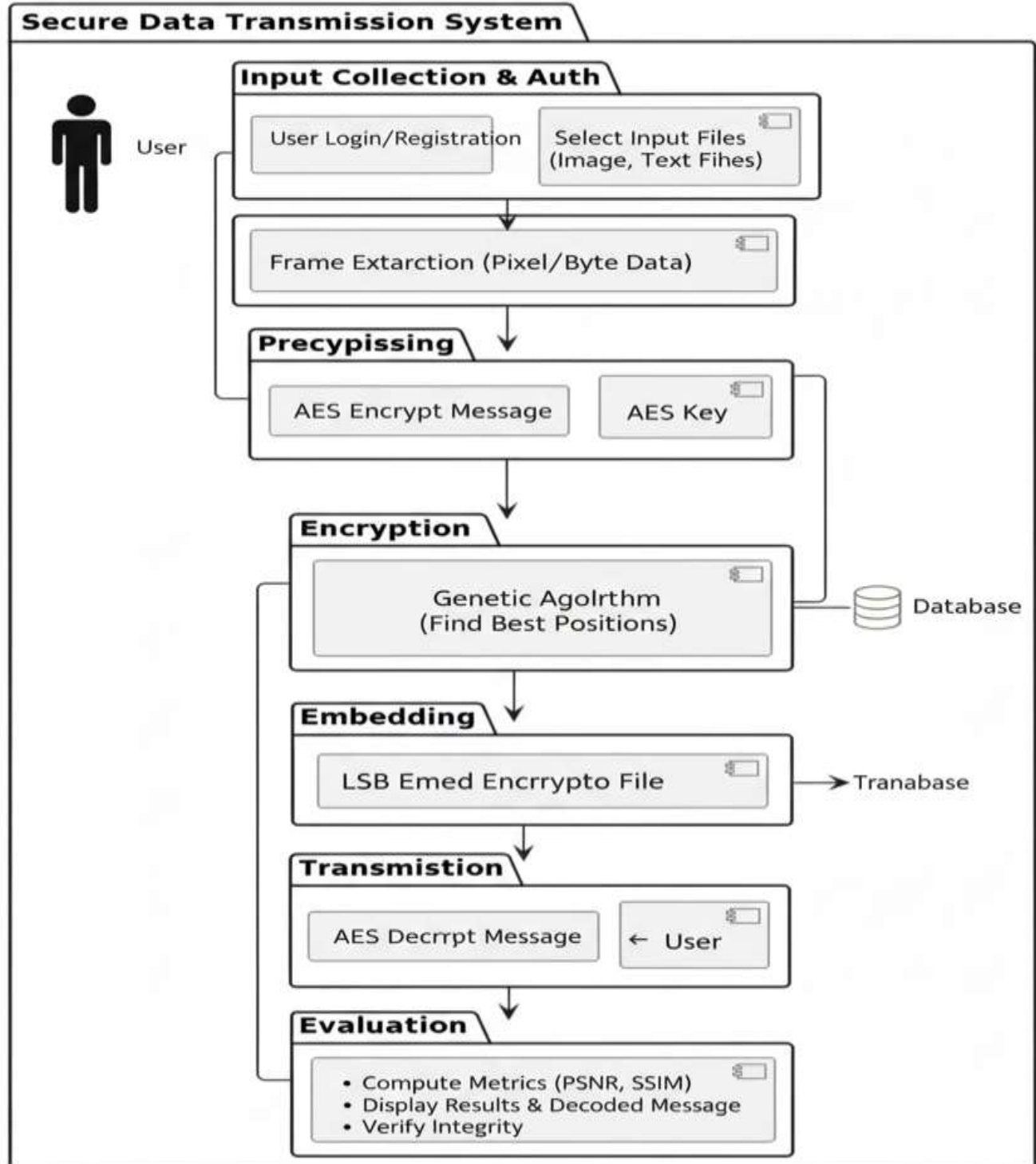


Fig.6.2.1 Architecture of Secure Data Transmission System

The proposed system for secure data transmission using AES Encryption, LSB Steganography, and Genetic Algorithm optimization follows a structured workflow that ensures confidentiality, robustness, and imperceptibility in the hidden data. The working flow begins with the input collection process and proceeds through various computational stages such as encryption, frame extraction, optimization, embedding, and decoding. Each phase of the system is interconnected, forming a complete and secure information-hiding pipeline. The detailed workflow of the system is described as follows.

System Working Flow Steps

The following steps describe the complete operational flow of the project:

Step 1: System Initialization and Input Collection

The system begins with an initialization phase where the user interacts with the application interface. The user is prompted to log in or register into the system using valid credentials. This ensures that only authorized users can access the data embedding and extraction functionalities, thereby maintaining system security.

After authentication, the user is allowed to select the input media files. The inputs generally consist of an image file, an audio file, and a secret text message that needs to be securely embedded.

- The image file serves as the cover object in which the secret information will be hidden.
- The audio file can also be used as an additional medium for embedding, depending on the configuration.
- The text file or plain text represents the confidential data to be transmitted secretly.

During this phase, all input files are validated to ensure they are in supported formats (e.g., .png, .jpg for images, .wav for audio). Any unsupported format is rejected by the system, and the user is prompted to upload valid input files again.

Step 2: Preprocessing and Frame Extraction

Once the input media is selected, the Frame Extraction Module is activated. In this module, the system reads the image or audio file and extracts its pixel or byte-level data. This step is essential for identifying the exact positions where the secret bits will be inserted.

For images, the frame extraction involves:

- Converting the image into a two-dimensional pixel matrix.
- Extracting the Red, Green, and Blue (RGB) components from each pixel.
- Storing these components in arrays for further processing.

For audio files, the process involves:

- Sampling the audio waveform and breaking it into frame blocks.
- Converting these blocks into byte streams.
- Preparing these bytes for modification during the LSB embedding process.

This extracted frame data provides the foundation for the subsequent steganography process, where the least significant bits (LSBs) will be modified to embed the secret information.

Step 3: AES Encryption of Secret Message

Before embedding, the AES Encryption Algorithm is applied to the secret text data to ensure the confidentiality of the hidden information. AES (Advanced Encryption Standard) is a symmetric encryption algorithm that converts plain text into unreadable ciphertext using a secret key.

The encryption process includes the following steps:

- Key Generation: The system generates a secure 128-bit or 256-bit AES key.
- Byte Padding: The text message is padded to ensure it fits evenly into the AES block size (16 bytes).

- Substitution-Permutation Steps: AES applies multiple rounds of substitution and permutation operations, including SubBytes, ShiftRows, MixColumns, and AddRoundKey transformations.
- Ciphertext Generation: The output of the AES algorithm is an encrypted binary string (ciphertext) that represents the secret message in a protected form.

The encrypted text is now secure and cannot be understood without the corresponding AES key. This adds a strong layer of cryptographic security to the steganography process.

Step 4: Genetic Algorithm Optimization

The Genetic Algorithm (GA) is implemented to optimize the embedding process. Its main objective is to select the most suitable positions (pixels or frames) for embedding the secret bits so that visual or auditory distortion is minimized.

The steps involved in this optimization phase are as follows:

1. Initial Population Generation: The algorithm randomly selects a set of potential embedding positions from the extracted frames.
2. Fitness Evaluation: Each candidate position is evaluated based on a fitness function that considers factors like image quality (PSNR), structural similarity (SSIM), and embedding capacity.
3. Selection: The best candidates (those with high fitness scores) are selected for the next generation.
4. Crossover and Mutation: Selected candidates are combined and slightly altered to explore better embedding positions.
5. Termination: The process repeats until the best embedding regions are determined.

The optimized embedding positions obtained through the GA are then passed to the LSB embedding module. This ensures that the hidden data has maximum imperceptibility and robustness against detection or manipulation.

Step 5: LSB Embedding Process

In this phase, the Least Significant Bit (LSB) algorithm is applied to hide the AES-encrypted bits into the cover image or audio. The LSB technique works by replacing the least significant bits of the cover media with bits from the encrypted message.

For example:

- Suppose the pixel value of an image is represented as 10101100.
- The secret bit to embed is 1.
- The new pixel value becomes 10101101, where only the last bit is modified.

This modification is visually undetectable because the change in pixel intensity is minimal. Similarly, for audio files, the amplitude bits of selected samples are modified slightly to embed the encrypted bits.

The LSB embedding continues until all encrypted bits of the secret message have been inserted into the cover object. Once completed, the system generates a new output file called the stego file, which visually or audibly appears identical to the original but secretly contains hidden data.

Step 6: Generation of Stego File and Transmission

After embedding, the system stores the final stego image or stego audio file. This file contains both the visible content and the hidden encrypted message. The generated stego file can then be transmitted to the intended receiver through secure channels such as email, cloud storage, or physical transfer.

Optionally, the system may include a Data Transmission Module integrated with APIs like Gmail or FTP, enabling automated sending of the stego file. Since the hidden data is encrypted with AES and further protected through LSB embedding, the overall transmission remains highly secure.

Step 7: Data Extraction and AES Decryption

At the receiver's end, the Decoding Module performs the reverse operation to recover the secret message. The process begins by reading the stego file and extracting the modified least significant bits from the pixels or audio frames. These bits are collected sequentially and reconstructed into the original encrypted binary string.

Once the entire encrypted message is recovered, the AES Decryption Module is invoked. The receiver must provide the same AES key that was used during encryption. The system applies the AES decryption process, reversing all substitution and permutation transformations, to obtain the original plain text.

If the correct AES key is not provided, the decryption process fails, ensuring that unauthorized users cannot access the hidden information.

Step 8: Evaluation Metrics

Finally, the system computes several evaluation metrics to assess the quality and accuracy of the steganography process:

- PSNR (Peak Signal-to-Noise Ratio): Measures the visual similarity between the cover and stego images.
- MSE (Mean Squared Error): Calculates the average squared difference between pixel values.
- SSIM (Structural Similarity Index): Evaluates structural distortion.
- Bit Error Rate (BER): Measures accuracy in bit extraction and recovery.

Higher PSNR and SSIM values indicate that the stego image retains its visual integrity and that the embedded data remains imperceptible to the human eye.

Step 9: Completion and Output Display

The final phase of the system displays the decoded secret message along with the evaluation results. The system confirms that the data transmission has been successfully completed with minimal distortion and maximum security. Both sender and receiver can verify the integrity of the transmission by comparing PSNR and MSE values.

This structured workflow demonstrates how cryptography (AES), steganography (LSB), and evolutionary optimization (GA) can be integrated to form a robust, efficient, and secure data-hiding framework. It ensures that confidential information can be transmitted safely without attracting attention or compromising privacy.

7. SOFTWARE TESTING

7.1 Software Testing

Software Testing Effective software testing ensures that the “Data Steganography for Image, Audio Using Deep Genetic Algorithm and LSB” system is reliable, secure, and performs accurately across all modules — from frame extraction and genetic-algorithm-based selection to AES encryption, LSB labeling, embedding, and final encoding and transmission.

Testing is critical for detecting defects, validating embedding/recovery accuracy, ensuring data integrity, and confirming that the system meets all functional and non-functional requirements defined in this SRS. The testing process was conducted systematically using multiple approaches to guarantee that the system performs correctly in both development and end-user environments.

1. Unit Testing

Purpose:

To verify that individual modules and functions of the system operate correctly in isolation.

What's Tested:

- Frame Extraction Module: Ensures correct extraction of image and audio frames and their conversion into byte arrays.
- Genetic Algorithm Operations: Validates that selection, crossover, and mutation functions correctly identify optimal frame positions for embedding.
- LSB Embedding Function: Checks that least significant bits are modified accurately for message insertion without corrupting the cover file.
- AES Encryption/Decryption: Confirms that plaintext messages are securely encrypted and decrypted using the correct key and block size.
- File I/O Functions: Verifies smooth reading and writing of image/audio files during encoding and decoding operations.

Tools Used:

- JUnit for testing individual Java classes and methods.
- Mockito for mocking file and encryption operations.
- NetBeans built-in JUnit runner for test management.

2. Integration Testing

Purpose:

To ensure that individual modules AES encryption, Genetic Algorithm, and LSB embedding work together seamlessly as one integrated system.

What's Tested:

- Data Flow: Verifies smooth data transfer between AES, GA, and LSB modules during encoding and decoding.
- File Handling: Ensures that encoded stego files are saved correctly and can be read accurately by the decoder.
- GUI and Backend Interaction: Confirms that the Swing interface properly communicates with backend modules and displays the correct results.
- Message Retrieval: Checks that the extracted message matches the original after decryption..

Tools Used:

- Integration test scripts developed in Java.
- Postman for API integration checks (if network features are enabled).
- File comparison utilities for verifying embedded and recovered data.

3. Functional Testing

Purpose:

To validate that all functional requirements defined in the SRS are implemented and perform as expected..

What's Tested:

- User Registration and Login: Ensures that users can register, authenticate, and access system functionalities.
- File Upload: Tests uploading of image/audio files and validates supported formats.
- Message Encryption and Embedding: Confirms successful encryption and embedding without exceeding file capacity.
- Message Decoding: Verifies correct extraction and decryption of hidden messages.
- Email Transmission: Checks successful sending and receiving of stego files through the integrated mailing module.

Tools Used:

- Manual functional testing using predefined test cases.
- Selenium for automating GUI-based workflows.
- File comparators for verifying encoded and decoded results.

4. User Acceptance Testing (UAT)

Purpose:

To ensure that the developed system meets the expectations and needs of the end users.

What's Tested:

- Ease of Use: Verifies that the interface is intuitive and easy to navigate for non-technical users.
- Workflow Accuracy: Checks that encoding, sending, and decoding processes follow the intended workflow.
- Message Integrity: Ensures the retrieved message matches the original text after transmission.
- Error Handling: Confirms that meaningful error messages are displayed for invalid inputs.

Tools Used:

- User feedback forms and practical usage trials.
- Structured UAT checklist to record observations.
- Manual GUI operation logs for performance feedback.

5. Security Testing

Purpose:

To confirm that the system protects data, prevents unauthorized access, and securely handles encryption keys

What's Tested:

- **AES Key Management:** Ensures encryption keys are stored securely and not logged in plain text.
- **Authentication:** Tests login verification and password protection mechanisms.
- **Data Confidentiality:** Verifies that embedded messages remain undetectable and encrypted.
- **Steganalysis Resistance:** Assesses system's resistance to statistical detection attacks.
- **Input Validation:** Tests handling of malformed inputs and file tampering.

Tools Used:

- OWASP ZAP for vulnerability scanning.
- Manual code reviews for security flaws.
- Custom PSNR/MSE scripts for evaluating data integrity.

6. Performance Testing

Purpose:

To evaluate the system's speed, resource usage, and efficiency under various workloads.

What's Tested:

- Embedding and Extraction Speed: Measures time taken to encode and decode files of different sizes.
- Encryption and Decryption Performance: Tests AES operation time for varying message lengths.
- Resource Utilization: Monitors CPU and memory usage during encoding and decoding.
- GUI Responsiveness: Ensures interface remains responsive during heavy processing tasks.

Tools Used:

- Apache JMeter for load simulation.
- Java VisualVM for memory and CPU profiling.
- Stopwatch-based benchmarking in NetBeans console

7. Regression Testing

Purpose:

To ensure that new modifications or updates do not introduce new errors or break existing features.

What's Tested:

- Re-Execution of Test Cases: Runs previous functional and integration tests after updates.
- Compatibility Checks: Ensures updates in encryption or GA logic do not affect decoding accuracy.
- GUI Stability: Verifies that all buttons, inputs, and outputs function correctly after revisions.

Tools Used:

- Automated JUnit regression suite.
- Git-based version control with test history tracking.
- Continuous Integration setup for repeated regression checks.

8. Usability Testing

Purpose:

To evaluate how effectively end users can interact with the system and understand its outputs.

What's Tested:

- Interface Simplicity: Ensures buttons, menus, and labels are clear and properly aligned.
- Feedback Clarity: Verifies success/failure messages are easily understandable.
- Accessibility: Confirms usability on different screen resolutions and systems.
- User Experience: Tests how smoothly a user can perform encoding, sending, and decoding tasks.

Tools Used:

- Manual observation and user interviews.
- GUI design review sessions.
- Heuristic evaluation checklist for visual and operational flow.

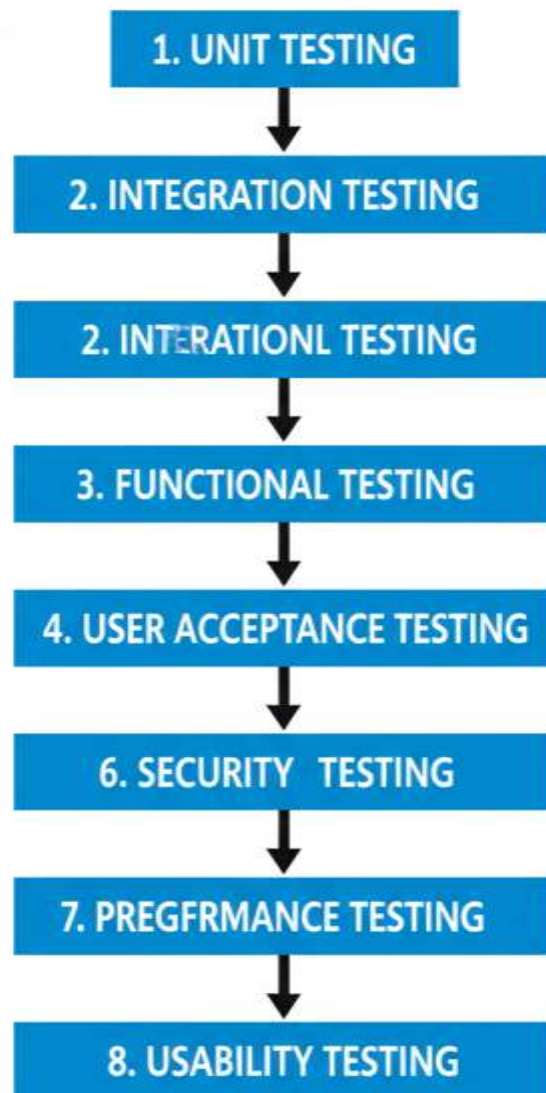


Fig 9.1 :- Software Testing Flow

8. RESULTS



The screenshot shows a window titled "Registration Frame" with a dark blue background. Inside, there is a registration form with the following fields and values:

Field	Value
Name	XYZ
Email Id	xyz@gmail.com
Mobile Number	9970426853
Username	xyz123
Password	*****
Confirm Password	*****

At the bottom of the form, there are three buttons: "Submit", "Clear", and "Back".

Fig. 8.1 Screenshot of Registration Frame



The screenshot shows a window titled "Data Steganography" with a dark blue background and a digital circuit pattern. Inside, there is a login form with the following fields and values:

Field	Value
Username	xyz123
Password	*****

At the bottom of the form, there are four buttons: "Login", "Registration", "Clear", and "Exit".

Fig. 8.2 Screenshot of Login Frame



Fig. 8.3 Screenshot of Operation Frame

A screenshot of a software window titled "Edit Profile Frame". The window has a menu bar with "Manage Profile", "Image", "Audio", and "Logout". The main content area is dark gray. In the center, there is a blue rectangular panel with the title "Edit Profile Frame" in white. Below the title, there are six input fields with labels to their left: "Name" (containing "xyz"), "Email Id" (containing "xyz@gmail.com"), "Mobile Number" (containing "5623897315"), "Username" (containing "xyz123"), "Password" (containing "*****"), and "Confirm Password" (containing "*****"). At the bottom of the blue panel, there are two white buttons labeled "Edit" and "Clear". The window includes standard Windows-style window controls in the top right corner.

Fig. 8.4 Screenshot of Edit Profile Frame

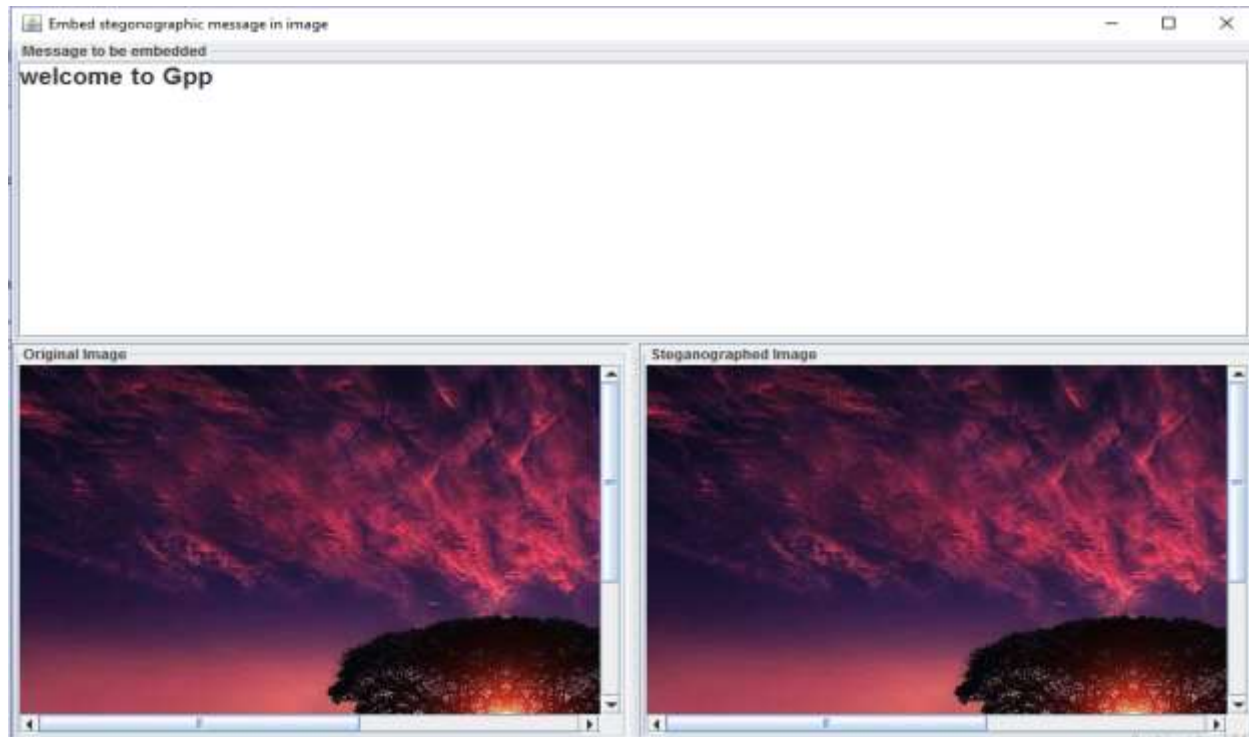


Fig. 8.5 Screenshot of Image Encoding Frame

9. ADVANTAGES & DISADVANTAGES

9.1 Advantages

- **Enhanced Data Security:** The integration of AES encryption with Genetic Algorithm-based frame selection provides a highly secure data-hiding process. This ensures that even if the steganographic layer is detected, the hidden data remains encrypted and inaccessible without the correct decryption key.
- **Improved Steganographic Efficiency:** The use of Deep Genetic Algorithms optimizes the selection of suitable frames and pixels for embedding, resulting in minimal distortion to the original audio or image file while maintaining a high payload capacity for hidden data.
- **High Imperceptibility and Robustness:** By applying Least Significant Bit (LSB) Labeling, the system conceals information in areas of minimal perceptual importance. This maintains the original file quality, making it nearly impossible for human vision or hearing to detect alterations.
- **Multi-Media Compatibility:** The proposed system supports both audio and image steganography, offering flexibility for users to hide text-based information in multiple media formats without loss of performance or reliability.
- **Efficient Frame Extraction and Reconstruction:** The frame extraction technique ensures efficient segmentation and embedding within the cover media, while frame unification accurately restores the final encoded audio or image file, preserving both structure and data integrity.

9.2 Disadvantages

- **Increased Computational Complexity:** The integration of Deep Genetic Algorithms and AES encryption increases the overall computation time compared to simple steganographic methods, especially when processing high-resolution images or long audio files.
- **Key Management Complexity:** Although AES encryption ensures higher security, managing and securely sharing encryption keys between sender and receiver requires additional precautions and may introduce operational challenges.
- **Limited Resistance to Compression Attacks:** If the stego file undergoes heavy compression (e.g., JPEG or MP3), some hidden data might become partially corrupted or unrecoverable due to bit modification in the compressed stream.
- **Higher Resource Utilization:** The use of multiple processing stages—encryption, optimization, embedding, and extraction—can increase memory and CPU usage on low-end systems, making it less suitable for lightweight devices.
- **No Real-Time Capability:** The system is designed primarily for secure data storage and transfer, not for real-time communication, as encoding and decoding require significant processing time.

10. CONCLUSION

The project “Data Steganography for Image, Audio Using Deep Genetic Algorithm and LSB” represents a significant advancement in the field of secure data transmission and information concealment. In an era of rapid digital communication, the risk of unauthorized access, data theft, and cybercrime continues to rise. This system offers an innovative solution by combining cryptography and steganography to ensure that confidential information can be transmitted securely and undetectably. By integrating AES encryption with Deep Genetic Algorithm-based frame optimization and Least Significant Bit (LSB) embedding, the project achieves a high level of data confidentiality, imperceptibility, and robustness. The proposed system hides sensitive data within digital images and audio files, ensuring that even if the communication channel is intercepted, the concealed data remains inaccessible without the correct encryption key. The Genetic Algorithm optimizes the embedding process by selecting the most suitable frames and pixel positions, thereby minimizing distortion and maintaining high fidelity of the cover media. Meanwhile, AES encryption provides a secure layer of protection, ensuring that the embedded information remains encrypted and secure against unauthorized decoding attempts.

REFERENCES

Reference Papers

- [1] Mahajan, Rikeeta, More, Bholenath, Gunjal, Lina, Waghulde, Bhavesh, & Narkhede, Rohit. (2024). Secure Data Transfer Over Internet Using Image Steganography. IJSART (International Journal for Science and Advanced Research in Technology), 10(2) *Secure data transfer over internet using image steganography*. IJSART, 10(2), IJSARTV10I280492..
- [2] Fan, Pingan, Zhang, Hong, & Zhao, Xianfeng. (2022). Robust video steganography for social media sharing based on principal component analysis. *EURASIP Journal on Information Security*, 2022, 10.1186/s13635-022-00130-z.
- [3] Mathur, H., & Veenadhari, S. (2018). Blended Vector Matrix on Different Channels of Image Encryption with Multi-Level Distinct Frequency Based Chaotic Approach to Prevent Cyber Crimes by Using Affine Transformation. (*ICICCT*), 650–656.
- [4] Rajkumar, G., & Malemath, V. (2017). Video Steganography: Secure Data Hiding Technique. *International Journal of Computer Network and Information Security*, 9, 38–45.
- [5] Kumar, S., Singh, N. K., Majumder, A., & Changder, S. (2018). A Novel Approach to Hide Text Data in Colour Image. *7th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, 577–581.

Reference Paper Links

- <https://ijsart.com/Home/IssueDetail/492>
- <https://doi.org/10.1186/s13635-022-00130-z>
- <https://doi.org/10.1109/ICICCT.2018.8473235>
- <https://doi.org/10.5815/ijcnis.2017.09.05>
- <https://doi.org/10.1109/ICRITO.2018.8748390>